```
"""
*********************
Student ID: w2149539
Date: 07/09/2025
*********************
"""
from graphics import *
import csv
import math

AIRPORT_CODES = {
    "LHR": "London Heathrow",
    "MAD": "Madrid Adolfo Suarez-Barajas",
    "CDG": "Charles De Gaulle International",
    "IST": "Istanbul Airport International",
    "AMS": "Amsterdam Schiphol",
    "LIS": "Lisbon Portela",
    "FRA": "Frankfurt Main",
    "FCO": "Rome Fiumicino",
    "MUC": "Munich International",
    "BCN": "Barcelona International"
}

AIRLINE_CODES = {
    "BA": "British Airways",
    "AF": "Air France",
    "AY": "Finnair",
    "KL": "KLM",
    "SK": "Scandinavian Airlines",
    "TP": "TAP Air Portugal",
    "TK": "Turkish Airlines",
    "W6": "Wizz Air",
    "U2": "easyJet",
    "FR": "Ryanair",
    "A3": "Aegean Airlines",
    "SN": "Brussels Airlines",
    "EK": "Emirates",
    "QR": "Qatar Airways",
    "IB": "Iberia",
    "LH": "Lufthansa"
}

data_list = []

def load_csv(csv_chosen):
    global data_list
    data_list.clear()
    try:
        with open(csv_chosen, 'r') as file:
            csvreader = csv.reader(file)
            header = next(csvreader)
            for row in csvreader:
                data_list.append(row)
        return True
    except FileNotFoundError:
        print(f"\nError: The file '{csv_chosen}' was not found.")
        return False
#task A
def get_valid_user_input():
    # Validate Airport Code
    while True:
        code = input("Please enter the three letter code for the departure city required: ").upper()
        if len(code) != 3:
            print("Wrong code length. please enter a three-letter city code")
        elif code not in AIRPORT_CODES:
            print("Unavailable city code please enter a valid city code")
        else:
            airport_code = code
            break

    # Validate Year
    while True:
        year_str = input("Please enter the year required in the format YYYY: ")
        if not year_str.isdigit() or len(year_str) != 4:
            print("Wrong data type. please enter a four-digit year value")
        else:
            year = int(year_str)
            if 2000 <= year <= 2025:
                selected_year = year
                break
            else:
                print("Out of range - please enter a value from 2000 to 2025")

    # Construct filename and return details
    filename = f"{airport_code}{selected_year}.csv"
    full_airport_name = AIRPORT_CODES[airport_code]
```

```
    return filename, full_airport_name, selected_year
#Task B
def calculate_outcomes(data):
    if not data:
        return None

    total_flights = len(data)
    runway_one_flights = 0
    over_500_miles = 0
    british_airways_flights = 0
    rain_departures = 0
    air_france_flights = 0
    delayed_departures = 0
    rainy_hours = set()
    destinations = []

    for row in data:
        # Each row is a list of strings from the CSV
        flight_num = row[1]
        scheduled_dep = row[2]
        actual_dep = row[3]
        destination_code = row[4]
        distance = int(row[5])
        runway = row[8]
        weather = row[9].lower()

        # Calculations based on row data
        if runway == '1':
            runway_one_flights += 1
        if distance > 500:
            over_500_miles += 1
        if flight_num.startswith('BA'):
            british_airways_flights += 1
        if flight_num.startswith('AF'):
            air_france_flights += 1
        if 'rain' in weather:
            rain_departures += 1
            rainy_hours.add(scheduled_dep[:2])
        if scheduled_dep != actual_dep:
            delayed_departures += 1

        destinations.append(destination_code)


    avg_departures_per_hour = round(total_flights / 12, 2)
    percent_air_france = round((air_france_flights / total_flights) * 100, 2) if total_flights > 0 else 0
    percent_delayed = round((delayed_departures / total_flights) * 100, 2) if total_flights > 0 else 0
    total_hours_of_rain = len(rainy_hours)


    if destinations:
        dest_counts = {dest: destinations.count(dest) for dest in set(destinations)}
        max_count = max(dest_counts.values())
        most_common_dest_codes = [code for code, count in dest_counts.items() if count == max_count]
        most_common_dest_names = [AIRPORT_CODES[code] for code in most_common_dest_codes]
    else:
        most_common_dest_names = ["N/A"]


    results = {
        "total_flights": total_flights,
        "runway_one_flights": runway_one_flights,
        "over_500_miles": over_500_miles,
        "british_airways_flights": british_airways_flights,
        "rain_departures": rain_departures,
        "avg_departures_per_hour": avg_departures_per_hour,
        "percent_air_france": percent_air_france,
        "percent_delayed": percent_delayed,
        "total_hours_of_rain": total_hours_of_rain,
        "most_common_destinations": most_common_dest_names
    }
    return results
#task C. Save Results as a Text File
def display_and_save_results(filename, airport_name, year, results):

    if results is None:
        print("No data to process.")
        return

    #output
    header = [
        "********************************************************************************",
        f"File {filename} selected - Planes departing {airport_name} {year}",
        "********************************************************************************"
    ]

    body = [
```

```
        f"The total number of flights from this airport was {results['total_flights']}",
        f"The total number of flights departing Runway one was {results['runway_one_flights']}",
        f"The total number of departures of flights over 500 miles was {results['over_500_miles']}",
        f"There were {results['british_airways_flights']} British Airways flights from this airport",
        f"There were {results['rain_departures']} flights from this airport departing in rain",
        f"There was an average of {results['avg_departures_per_hour']} flights per hour from this airport",
        f"Air France planes made up {results['percent_air_france']}% of all departures",
        f"{results['percent_delayed']}% of all departures were delayed",
        f"There were {results['total_hours_of_rain']} hours in which rain fell",
        f"The most common destinations are {results['most_common_destinations']}"
    ]

    output_block = "\n".join(header + body)

    print("\n" + output_block)

    # Task C: Save to results.txt in append mode ('a')
    with open("results.txt", "a") as f:
        f.write(output_block + "\n\n")

#task D Histogram
def plot_histogram(data, airport_name, year):

     # Add check for empty data
    if not data:
        print("No data available to plot histogram.")
        return
    # Get and validate airline code
    while True:
        airline_code = input("\nEnter a two-character Airline code to plot a histogram: ").upper()
        if airline_code in AIRLINE_CODES:
            break
        else:
            print("Unavailable Airline code please try again.")

    airline_name = AIRLINE_CODES[airline_code]

    hourly_counts = [0] * 12
    for row in data:
        if row[1].startswith(airline_code):
            hour = int(row[2][:2])
            if 0 <= hour < 12:
                hourly_counts[hour] += 1

    # Set up the graphics window
    win_width = 800
    win_height = 600
    win = GraphWin("Histogram", win_width, win_height)
    win.setBackground("gray95")

    # Determine scale based on the max number of flights in any hour
    max_flights = max(hourly_counts) if hourly_counts else 1

    #  create margins and scale the Y-axis
    win.setCoords(-1.5, -0.2 * max_flights, 12.5, 1.2 * max_flights)

    # Draw Title
    title_text = f"DEPARTURES BY HOUR FOR {airline_name} FROM {airport_name} {year}"
    title = Text(Point(5.5, 1.05 * max_flights), title_text)
    title.setSize(16)
    title.setStyle("bold")
    title.draw(win)

    # Draw X-axis Label
    xaxis_label = Text(Point(5.5, -0.1 * max_flights), "Hours 00:00 to 11:59")
    xaxis_label.draw(win)

    # Draw bars, hour labels, and count labels
    for i, count in enumerate(hourly_counts):
        # Hour label on X-axis
        hour_label = Text(Point(i, -0.05 * max_flights), f"{i:02d}")
        hour_label.draw(win)

        if count > 0:
            # Create and draw the bar (a rectangle)
            bar = Rectangle(Point(i - 0.4, 0), Point(i + 0.4, count))
            bar.setFill("lightgreen")
            bar.setOutline("darkgreen")
            bar.draw(win)

            # Create and draw the numerical value on top of the bar
            count_label = Text(Point(i, count + 0.05 * max_flights), str(count))
            count_label.setStyle("bold")
            count_label.draw(win)

    # Wait for a mouse click to close the window
    win.getMouse()
```

```
    win.close()

#Task E )Program Loops on Request and Loads a New CSV file
def main():

    while True:
        # Task A: Get and validate user input
        filename, airport_name, year = get_valid_user_input()

        # Display selection confirmation
        print("\n**********************************************************************")
        print(f"File {filename} selected - Planes departing {airport_name} {year}.")
        print("**********************************************************************")

        # Load the selected CSV file
        if not load_csv(filename):
            continue

        # Task B :Calculate outcomes
        results = calculate_outcomes(data_list)

        #Display and save the results
        display_and_save_results(filename, airport_name, year, results)

        # Task D: Plot the histogram
        plot_histogram(data_list, airport_name, year)

        # Task E
        while True:
            rerun = input("\nDo you want to select a new data file? Y/N: ").upper()
            if rerun in ['Y', 'N']:
                break
            else:
                print("Invalid input. Please enter 'Y' or 'N'.")

        if rerun == 'N':
            print("\nThank you. End of run")
            break # Exit main while loop

if __name__ == "__main__":
    main()
```